
Première partie (10 points) -> copie n°1

Exercice 1 – COURS, Définitions

- 1.1 – Quelles sont les 4 étapes du cycle de vie des fichiers en Python ? Pour chaque étape, citer au moins une fonction.
- 1.2 – Dans `def ma_fonction(*args, **kwargs)`, à quoi correspondent `*args` et `**kwargs` ? Donner le type de ces deux attributs.
- 1.3 – Qu'est-ce qu'un décorateur et à quoi sert-il ? Agit-il sur le code de la fonction décorée ?
- 1.4 – Quelle est la différence entre une liste et un set ? Et entre une liste et un tuple ?
- 1.5 – Quelle est la différence entre `type(obj) == Type` et `isinstance(obj, Type)` ?

Exercice 2 – CROOSH, la billetterie nouvelle génération

Votre prochain événement a lieu dans 1 mois mais il semble qu'il manque quelque chose d'essentiel : une billetterie digne de ce nom. Votre objectif : coder une billetterie très simple (mais pas très efficace) qui soit réutilisable.

Structure d'un événement

Un événement est représenté par un dictionnaire contenant les (clés : valeurs) suivantes :

- **prix** : le prix d'une place
- **participants** : La liste des logins des participants déjà inscrits à l'événement
- **nb_places_max** : Le nombre de places disponibles pour l'événement

2.1 : Écrire une fonction `creer_evt()` qui initialise et retourne un événement en utilisant la structure définie ci-dessus. La liste des participants est vide lors de la création. On vérifiera que le nombre de places maximal est un entier positif non nul.

Désormais, votre programme va gérer l'ensemble des événements dans un dictionnaire dont les clés seront les noms des événements et les valeurs les événements (sous forme de dictionnaire tel que défini ci-dessus).

2.2 : Écrire une fonction `retirer_evt()` qui prend en paramètres le dictionnaire contenant tous les événements et le nom de l'événement à supprimer.

2.3 : Écrire une fonction `ajouter_participant()` qui prend en paramètres le dictionnaire contenant tous les événements, le nom de l'événement et le login du participant à ajouter. Cette fonction lèvera une exception si le login du participant n'est pas une chaîne de caractères de longueur inférieure à 8 caractères sans espaces. Elle lèvera également une exception de type `EventFullError` s'il n'y a plus de places.

2.4 : Écrire une fonction `stats_ventes()` qui prend en paramètre le dictionnaire contenant tous les événements et retourne un nouveau dictionnaire dont les clés sont les noms des événements et les valeurs sont le montant total des ventes pour l'événement (`prix*nombre de places vendues`).

2.5 : Écrire un programme qui crée 2 événements avec 1 participant dans chaque. Le programme devra gérer les exceptions potentiellement levées par votre code.

Deuxième partie (10 points) changer de copie -> copie n°2

Exercice 3 : initiation au HTML

Introduction Le web est créé en 1989 par Tim Berners Lee, informaticien au CERN. Les pages web utilisent différents langages pour décrire leur contenu, leur style et leur comportement - respectivement HTML, CSS et JavaScript. Dans la figure 1 au verso, on donne un exemple de document simplifié HTML (à gauche). Un navigateur web (Firefox, Chrome...) produira le résultat illustré à droite à quelques détails près, pour simplifier l'exercice.

L'objectif de cet exercice est de gérer des documents HTML grâce aux concepts de la programmation orientée objet.

```

<html>
  <h1>
    Je suis un titre !
  </h1>
  <p style="color:blue">
    Bout de paragraphe bleu.
    <b>
      Bout de paragraphe écrit en gras !
    </b>
  </p>
  
  <b>
    Bout de paragraphe écrit en gras !
  </b>
</html>

```



Figure 1 : exemple de code HTML (à gauche) et de résultat affiché par un navigateur web (à droite)

HTML en 5 minutes : Un document HTML est formé de **balises** pouvant contenir d'autres balises. Une balise :

- A un **nom** qui définit sa fonction toujours en minuscule (html, h1, p, b, img). Le nom sert dans la balise ouvrante <nom>
 - Peut avoir des **attributs** optionnels sous la forme d'un dictionnaire {"nom_attribut": "valeur_attribut"}. Ces attributs seront indiqués dans la balise ouvrante juste après le nom : <nom attribut1="valeur1" attribut2="valeur2" ...>
 - Peut avoir du contenu. Dans ce cas, la balise se ferme avec la syntaxe </nom>
- Le **contenu**, peut être un mélange de texte simple et/ou d'autres balises ayant elles-mêmes du contenu. Le contenu est positionné entre <nom ...> et </nom>.
- Exemple de balise avec un attribut et avec du contenu **texte** simple : <nom attribut="valeur"> **texte** </nom>
 - Exemple de balise sans attribut, avec du contenu composé d'un **texte** et d'une autre balise **interne** ayant aussi du contenu texte : <nom> **texte** <interne> **texte_interne** </interne> </nom>
 - Les balises dites vides n'ont pas de contenu ni de balises fermantes. Exemple : <nom attribut="valeur" >.
 - Un document HTML commence toujours par une balise <html>.

On propose de représenter les balises par des classes Python en utilisant les mécanismes d'héritage vu en cours.

Soit les classes suivantes :

- **Balise** représente toutes les balises (vides et non vides) : elle contient donc un nom en minuscule et des attributs (optionnels) sous forme de dictionnaire : {"nom_attribut": "valeur_attribut"}.
- **BaliseContenu** est une balise auquel on ajoute du contenu (balise non vide). En plus du nom et des attributs (optionnels), elle a un contenu composé d'un ou plusieurs éléments, chaque élément est soit un texte (chaîne de caractères) soit une balise (vide ou non vide).

3.1 Écrire ces 2 classes avec leur constructeur. Implémenter les getter (@property), et les setter quand cela est pertinent. Pour les balises avec contenu, on vérifiera que le contenu est composé de textes et/ou de balises. Si ce n'est pas le cas, lever une exception.

Soit les classes suivantes et leurs balises correspondantes (voir l'exemple de la figure 1):

Classe	Balise
Html	<html> </html>
Titre	
Paragraphe	
Gras	
Image	 (balise vide)

3.2 Écrire ces classes. Pour la classe **Image**, vérifier qu'il existe un attribut contenant le chemin de l'image (attribut `src`) et que ce chemin existe (`os.path.exists()`). Si c'est le cas, on définira un attribut d'instance (avec getter et setter) contenant ce chemin, sinon, on déclenche une exception. Si une classe vous semble très similaire à une autre, vous pouvez écrire uniquement leur(s) différence(s) plutôt que de l'écrire intégralement.

3.3 On veut maintenant générer du texte correspondant au code HTML à partir des objets définis ci-dessus. Écrire la ou les méthodes `__str__()` nécessaire(s) pour pouvoir afficher du code HTML, en précisant la ou les classe(s) associée(s). Dans ce code on gère les retours à la ligne (`\n`) mais on ne gère pas les tabulations visibles dans l'exemple.

3.4 En supposant que toutes les classes sont écrites, écrire une fonction `main()` permettant d'afficher le code HTML de la Figure 1 à l'aide des classes créées à la question 3.2.

3.5 On voudrait aussi connaître le nombre de balises (vides ou non vides) affichées dans le code HTML. Par exemple il y a 6 balises affichées (dont 2 fois la même) dans le code de la Figure 1. Modifier la classe **Balise** pour gérer ce nombre et ajouter dans le `main()` l'affichage du nombre de balises.